Manual de Usuario — 3xtrat3rr3str3D

Guía práctica para instalar y utilizar el compilador, editar programas en Lat, Y? y Zetariano, generar ensamblador RISC-V y ejecutar el resultado desde la consola integrada.

1. ¿Qué es 3xtrat3rr3str3D?

Es un compilador para tres lenguajes de alto nivel de la Resistencia:

Lenguaje	**Extensión**	**Estilo**
Lat (Pig Latin)	.pig	Imperativo; permite importar los otros dos lenguajes
Y?	.y	Indentación significativa, estructuras y funciones
Zetariano	.z	Orientado a objetos, con una clase por archivo

El compilador valida el programa, genera código de tres direcciones (C3D) y lo traduce a un archivo de ensamblador RISC-V (.s). La interfaz permite ejecutar ese archivo mediante RARS y escribir los datos que solicite el programa.

Zetariano incluye herencia simple, encapsulamiento, sobrescritura con @Override, this y llamadas al constructor padre con super(...).

2. Requisitos e instalación

2.1 Requisitos

- Java 17.
- Maven 3.9.x.
- Un entorno gráfico para abrir la aplicación Swing.
- El archivo lib/rars1_6.jar, incluido con el proyecto, para ejecutar el ensamblador.
- Opcionalmente, un IDE como IntelliJ IDEA.

Puedes comprobar las herramientas desde una terminal:

```
java -version
mvn -version
```

2.2 Preparar el proyecto

Descarga o clona el repositorio y abre su carpeta raíz. En IntelliJ IDEA, importa el proyecto Maven y selecciona Java 17 como SDK. Conserva la carpeta lib/ junto con la aplicación.

2.3 Compilar la aplicación

Desde la carpeta que contiene pom.xml:

```
mvn -q compile
```

Maven descargará las dependencias necesarias y compilará el proyecto.

3. Inicio rápido

3.1 Abrir la interfaz

En IntelliJ IDEA, ejecuta la clase com.navi.Main. La ventana se abre directamente.

En Linux también puedes iniciarla desde la raíz del proyecto con:

```
mvn -q compile
CP="target/classes:$(mvn -q dependency:build-classpath -Dmdep.outputFile=/dev/stdout)"
java -cp "$CP" com.navi.Main
```

3.2 Compilar un ejemplo

1. Selecciona **Archivo** → **Abrir proyecto...**.
2. Abre la carpeta testfile/ del repositorio.
3. Haz doble clic en main.pig.
4. Pulsa **Compilar** para analizarlo y generar testfile/output/main.s.
5. Consulta las pestañas de resultados.
6. Pulsa **Compilar y ejecutar** o F5 para generar nuevamente el ensamblador y ejecutarlo en la consola integrada.

El ejemplo importa clases de testfile/z/ y muestra el uso de referencias de tipo padre con objetos de clases hijas.

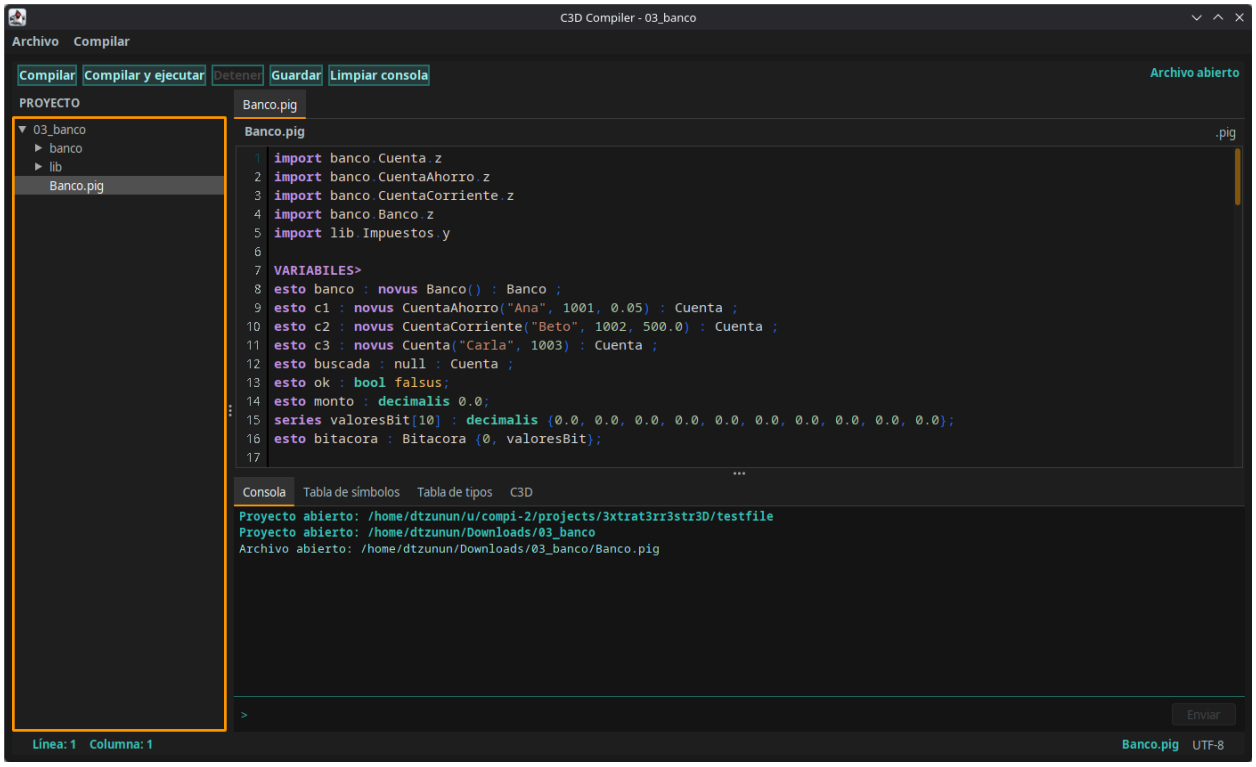
4. Tour de la interfaz

La distribución conserva el explorador a la izquierda, el editor al centro y los resultados en la parte inferior.

Componente	Función
Compilar	Valida el archivo activo y genera el ensamblador
Compilar y ejecutar	Compila y, si tiene éxito, inicia el programa con RARS
Detener	Finaliza la ejecución activa
Guardar	Guarda el archivo activo
Limpiar consola	Borra el texto mostrado en la consola
Explorador de proyecto	Permite acceder a carpetas y archivos
Editor	Abre cada archivo en una pestaña
Barra de estado	Muestra línea, columna, archivo y codificación

Las pestañas de resultados son:

- **Consola:** mensajes de compilación, errores, salida del programa y campo para enviar datos.
- **Tabla de símbolos:** variables, parámetros y elementos registrados, con tipo, ámbito y posición de memoria.
- **Tabla de tipos:** clases y estructuras con sus miembros.
- **C3D:** cuartetas generadas, con números de línea.



5. Gestión del árbol de trabajo

5.1 Abrir un proyecto

Selecciona **Archivo** → **Abrir proyecto...** y elige la carpeta de tus fuentes. El explorador cargará su estructura y el título de la ventana mostrará el nombre del proyecto.

5.2 Abrir un archivo

Haz doble clic sobre el archivo en el explorador. Se abrirá en una pestaña del editor. Si ya estaba abierto, se seleccionará su pestaña.

5.3 Crear archivos y carpetas

1. Haz clic derecho sobre una carpeta del explorador.
2. Selecciona **Nuevo archivo** o **Nueva carpeta**.
3. Escribe el nombre.
4. Para fuentes del compilador, utiliza las extensiones .pig, .y o .z.

Para una clase Z, utiliza un archivo con el nombre de la clase, por ejemplo Persona.z.

5.4 Eliminar

Haz clic derecho sobre el elemento y selecciona **Eliminar**. Revisa la selección antes de confirmar: si es una carpeta, se elimina su contenido. Las pestañas asociadas a archivos eliminados se cierran.

5.5 Guardar

Guardar o Ctrl+S guarda el archivo activo. Al compilar, la aplicación guarda automáticamente ese archivo. Guarda también los módulos importados que hayas modificado antes de compilar el programa principal, ya que se cargan desde el disco.

6. Editar código y resaltado

El editor distingue visualmente palabras reservadas, tipos, booleanos, números, cadenas, caracteres, comentarios, operadores e identificadores. Los números de línea ayudan a localizar los errores reportados.

Acción	**Atajo o gesto**
Guardar archivo activo	Ctrl+S
Deshacer	Ctrl+Z
Rehacer	Ctrl+Shift+Z o Ctrl+Y
Insertar cuatro espacios	Tab
Compilar y ejecutar	F5
Cerrar una pestaña	Clic central sobre ella
Cerrar otras o todas las pestañas	Clic derecho sobre una pestaña

En Y, mantén una indentación consistente: los espacios al comienzo de las líneas forman parte de la estructura del programa.

7. Compilar, ejecutar y consultar resultados

7.1 Compilar

1. Selecciona el archivo que deseas compilar. Para un programa con varios módulos, utiliza su .pig principal.

2. Pulsa **Compilar** o **Compilar → Compilar archivo actual**.
3. Consulta la consola.

Si el proceso termina correctamente, aparecerán mensajes como:

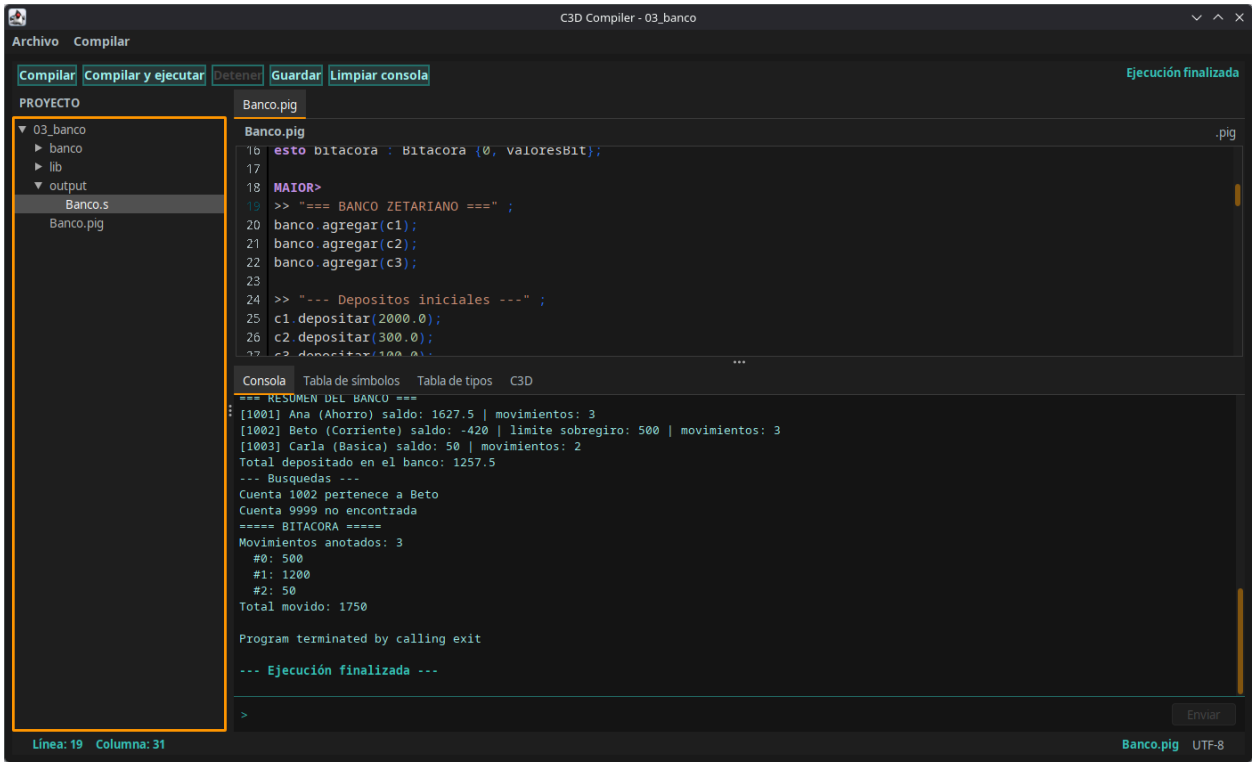
Análisis semántico completado.

Código de tres direcciones generado.

Archivo RISC-V: .../output/main.s

COMPILACIÓN EXITOSA

Las tablas y la pestaña C3D se llenan con los resultados de esa compilación. Si hay errores, se muestran en la consola y se deben corregir antes de generar una nueva salida válida.



7.2 Ubicación del ensamblador

El archivo se guarda en una carpeta output/ junto al fuente compilado. Por ejemplo:

Archivo compilado	Archivo generado
testfile/main.pig	testfile/output/main.s
miProyecto/inicio.pig	miProyecto/output/inicio.s

Al generar una nueva salida, la aplicación limpia los archivos regulares de esa carpeta. Reserva output/ para los resultados del compilador.

7.3 Compilar y ejecutar

Pulsa **Compilar y ejecutar** o F5. Si la compilación es correcta, se inicia RARS y aparece:

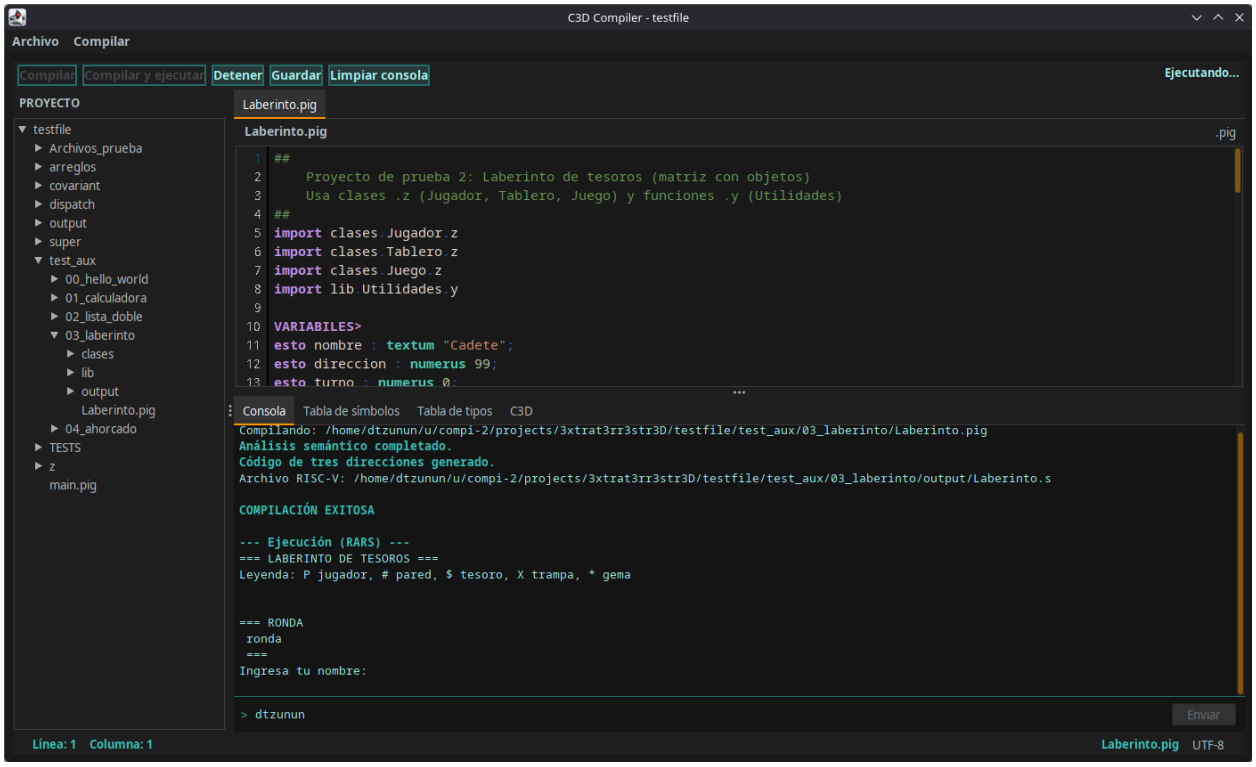
--- Ejecución (RARS) ---

A continuación se muestra la salida del programa. Cuando termina normalmente aparece **Ejecución finalizada**. Durante el proceso se deshabilitan los botones para iniciar otra compilación.

7.4 Ingresar datos al programa

1. Espera a que el programa solicite un valor.
2. Escribe el dato en el campo inferior de la pestaña **Consola**, junto al símbolo >.
3. Presiona Enter o pulsa **Enviar**.
4. Repite el procedimiento para cada lectura.

Escribe el tipo de dato que espera el programa. El campo de entrada está habilitado durante la ejecución; el texto de salida de la consola es de solo lectura.

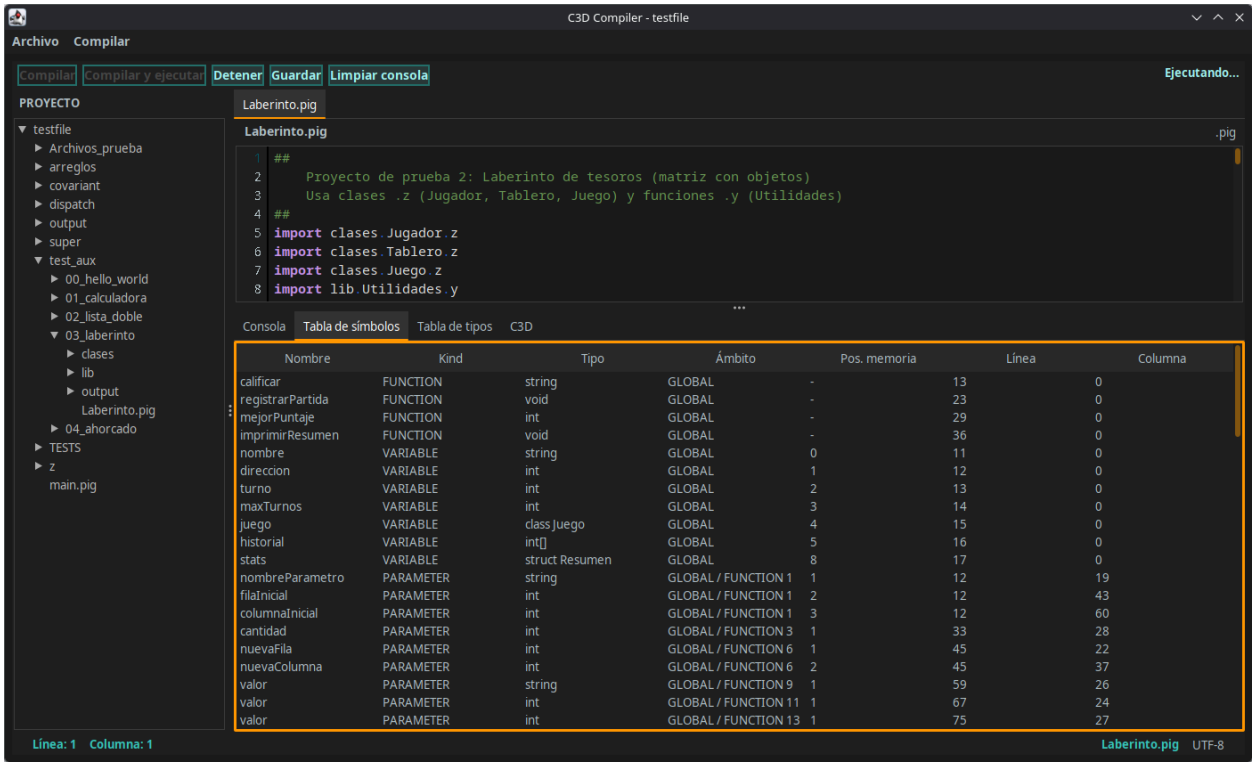


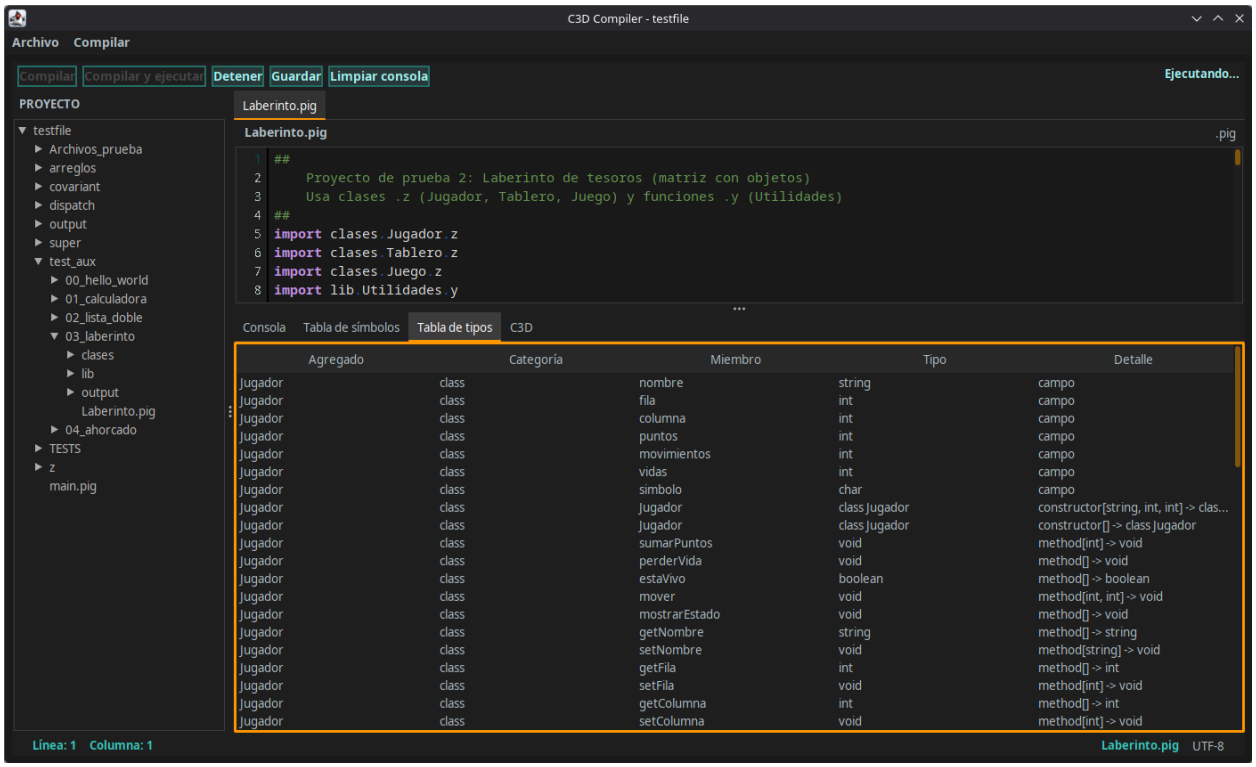
7.5 Detener

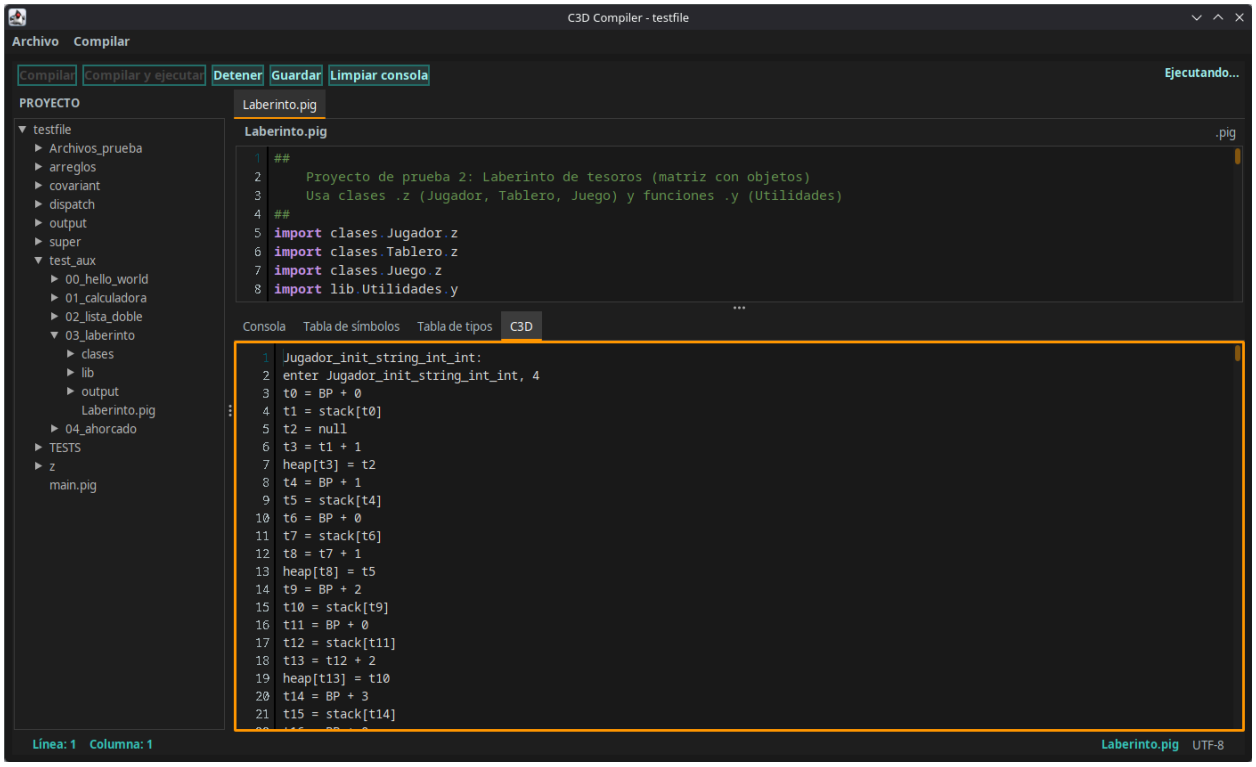
Pulsa **Detener** para finalizar el programa activo, por ejemplo si está esperando una entrada o ejecutando un ciclo. La consola mostrará **Ejecución detenida** y los controles volverán a quedar disponibles. **Limpiar consola** únicamente borra el texto; no detiene el programa.

7.6 Consultar las tablas y el C3D

Selecciona **Tabla de símbolos** para revisar nombres, tipos y ámbitos; **Tabla de tipos** para revisar estructuras y clases; y **C3D** para consultar la traducción intermedia.


C3D Compiler - testfile
Archivo Compilar
Compilar Compilar y ejecutar Detener Guardar Limpiar consola
Ejecutando...
PROYECTO
testfile
Archivos_prueba
arreglos
covariant
dispatch
output
super
test_aux
00_hello_world
01_calculadora
02_lista_doble
03_laberinto
clases
lib
output
Laberinto.pig
04_ahorcado
TESTS
z
main.pig
Laberinto.pig
Laberinto.pig
.pig
1 ##
2 Proyecto de prueba 2: Laberinto de tesoros (matriz con objetos)
3 Usa clases .z (Jugador, Tablero, Juego) y funciones .y (Utilidades)
4 ##
5 import clases.Jugador.z
6 import clases.Tablero.z
7 import clases.Juego.z
8 import lib.Utilidades.y
Consola Tabla de símbolos Tabla de tipos C3D
Nombre Kind Tipo Ámbito Pos. memoria Línea Columna
calificar FUNCTION string GLOBAL - 13 0
registrarPartida FUNCTION void GLOBAL - 23 0
mejorPuntaje FUNCTION int GLOBAL - 29 0
imprimirResumen FUNCTION void GLOBAL - 36 0
nombre VARIABLE string GLOBAL 0 11 0
direccion VARIABLE int GLOBAL 1 12 0
turno VARIABLE int GLOBAL 2 13 0
maxTurnos VARIABLE int GLOBAL 3 14 0
juego VARIABLE class Juego GLOBAL 4 15 0
historial VARIABLE int[] GLOBAL 5 16 0
stats VARIABLE struct Resumen GLOBAL 8 17 0
nombreParametro PARAMETER string GLOBAL / FUNCTION 1 1 12 19
filaInicial PARAMETER int GLOBAL / FUNCTION 1 2 12 43
columnaInicial PARAMETER int GLOBAL / FUNCTION 1 3 12 60
cantidad PARAMETER int GLOBAL / FUNCTION 3 1 33 28
nuevaFila PARAMETER int GLOBAL / FUNCTION 6 1 45 22
nuevaColumna PARAMETER int GLOBAL / FUNCTION 6 2 45 37
valor PARAMETER string GLOBAL / FUNCTION 9 1 59 26
valor PARAMETER int GLOBAL / FUNCTION 11 1 67 24
valor PARAMETER int GLOBAL / FUNCTION 13 1 75 27
Línea: 1 Columna: 1
Laberinto.pig UTF-8


C3D Compiler - testfile
Archivo Compilar
Compilar Compilar y ejecutar Detener Guardar Limpiar consola
Ejecutando...
PROYECTO
testfile
Archivos_prueba
arreglos
covariant
dispatch
output
super
test_aux
00_hello_world
01_calculadora
02_lista_doble
03_laberinto
clases
lib
output
Laberinto.pig
04_ahorcado
TESTS
z
main.pig
Laberinto.pig
Laberinto.pig
.pig
1 ##
2 Proyecto de prueba 2: Laberinto de tesoros (matriz con objetos)
3 Usa clases .z (Jugador, Tablero, Juego) y funciones .y (Utilidades)
4 ##
5 import clases.Jugador.z
6 import clases.Tablero.z
7 import clases.Juego.z
8 import lib.Utilidades.y
Consola Tabla de símbolos Tabla de tipos C3D
Agregado Categoría Miembro Tipo Detalle
Jugador class nombre string campo
Jugador class fila int campo
Jugador class columna int campo
Jugador class puntos int campo
Jugador class movimientos int campo
Jugador class vidas int campo
Jugador class simbolo char campo
Jugador class Jugador class Jugador constructor[string, int, int] -> clas...
Jugador class Jugador class Jugador constructor[] -> class Jugador
Jugador class sumarPuntos void method[int] -> void
Jugador class perderVida void method[] -> void
Jugador class estaVivo boolean method[] -> boolean
Jugador class mover void method[int, int] -> void
Jugador class mostrarEstado void method[] -> void
Jugador class getNombre string method[] -> string
Jugador class setNombre void method[string] -> void
Jugador class getFila int method[] -> int
Jugador class setFila void method[int] -> void
Jugador class getColumna int method[] -> int
Jugador class setColumna void method[int] -> void
Línea: 1 Columna: 1
Laberinto.pig UTF-8



7.7 Ejecutar el archivo desde una terminal

También puedes ejecutar el ensamblador generado desde la raíz del repositorio:

java -jar lib/rars1_6.jar nc me testfile/output/main.s

La entrada y salida se realizan en esa terminal. El archivo .s es código para el simulador RARS; no se ejecuta directamente como un programa nativo del sistema operativo.

8. Los tres lenguajes

8.1 Lat — .pig

Un programa Lat puede tener variables globales, funciones y una sección principal obligatoria MAIOR>. Se cierra con FINIS;.

VARIABILES>
esto edad : numerus 20;
esto nombre : textum "Cadete";
MAIOR>
>> "Hola " >> nombre;
si (edad >= 18) {

```
   >> "Es mayor de edad";
} aliter {
   >> "Es menor de edad";
} finis;
FINIS;
```

Puntos clave:

- Variable: esto nombre : tipo valor;.
- Arreglo: series datos[3] : numerus {1, 2, 3};.
- Imprimir: >> expresion;. Leer en una variable: variable <<.
- Función sin retorno: actio nombre(parametros) { ... } finis;.
- Función con retorno: ratio tipo nombre(parametros) { ... } finis;.
- Variables locales de funciones: se declaran al inicio mediante VARIABILES [...].
- Importación: import carpeta.Archivo.z o import carpeta.archivo.y, sin punto y coma.
- Parámetro arreglo: series matriz[][] : numerus.

Para trabajar con objetos y herencia:

esto objeto : novus Hija() : Padre;

esto pendiente : null : Padre;

La primera variable es una referencia de tipo Padre, pero contiene un objeto Hija. Al llamar a un método sobrescrito se ejecuta la versión de la clase real del objeto.

8.2 Y? — .y

La indentación delimita los bloques. %estructuras es opcional y %funciones es obligatoria.

```
%estructuras
estructura Punto:
   entero x
   entero y

%funciones
definir sumar(entero a, entero b) -> entero:
   retornar a + b

definir main():
   entero resultado = sumar(3, 4)
   imprimir("Resultado: ", resultado)
```

Puntos clave:

- Función: definir nombre(parametros) -> tipo:; el retorno puede omitirse.
- Condicional: si (condicion) entonces, sino (condicion) entonces y contrario.
- Ciclos: para, mientras y hacer.
- Control: romper, continuar y retornar.
- Entrada y salida: leer() e imprimir(...).
- Parámetros arreglo: []entero datos o [][]entero matriz.
- Parámetro estructura por referencia: {} Punto punto.

8.3 Zetariano — .z

Cada archivo contiene una clase. Este ejemplo corresponde a Persona.z:

```
public class Persona {
   private String nombre;
   protected int edad;

   public Persona(String nombre, int edad) {
      this.nombre = nombre;
      this.edad = edad;
   }

   public void saludar() {
      println("Hola, me llamo " + this.nombre);
   }

   public int anioNacimiento(int actual) {
      return actual - this.edad;
   }
}
```

Los modificadores controlan el acceso:

Modificador	**Desde dónde se puede acceder**
public	Desde cualquier clase y desde Lat
private	Solo desde la propia clase
protected	Desde la propia clase y las clases hijas
Sin modificador	Solo desde la propia clase, como private

this hace referencia al objeto actual. En this.nombre = nombre, el lado izquierdo es el atributo y el derecho es el parámetro.

Se conservan métodos y constructores sobrecargados, arreglos con new, referencias null, condicionales, ciclos, selección múltiple, print, println y readln.

8.4 Herencia, @Override y super

Una clase hija se declara con extends. Por ejemplo, en Profesor.z:

```
public class Profesor extends Persona {

    public Profesor(String nombre, int edad) {

        super(nombre, edad);

    }

    @Override

    public void saludar() {

        println("Soy profesor.");

    }

}
```

super(nombre, edad); llama al constructor de Persona y debe ser la primera instrucción del constructor hijo. Si se omite, el padre debe disponer de un constructor sin argumentos accesible. Una clase sin constructores declarados recibe uno público sin argumentos.

@Override es obligatorio al sobrescribir un método heredado. Deben mantenerse los tipos de parámetros; el retorno debe ser compatible y la visibilidad no puede reducirse. Colocar la anotación en un método que no sobrescribe otro también es un error.

Para usar las dos clases desde main.pig, importa ambos archivos y declara el objeto en la sección de variables:

```
import Persona.z

import Profesor.z
```

VARIABILES>

esto persona : novus Profesor("Ana", 35) : Persona;

esto pendiente : null : Persona;

MAIOR>

persona.saludar();

FINIS;

Al ejecutarlo se imprime Soy profesor.. Aunque la variable está declarada como Persona, se utiliza el método sobrescrito de Profesor.

Z permite una sola clase padre por clase y no utiliza paquetes. Importa primero la clase padre y después sus clases hijas, como en el ejemplo. Para acceder desde Lat a un miembro o constructor, este debe ser público.

8.5 Tabla comparativa rápida

Concepto	**Lat**	**Y**	**Z**
Entero	numerus	entero	int
Decimal	decimalis	flotante	double
Carácter	littera	caracter	char
Cadena	textum	cadena	String
Booleano	bool	bool	boolean
Verdadero / falso	verum / falsus	verdadero / falso	true / false
Imprimir	>> x;	imprimir(x)	print(x); o println(x);
Leer	x <<	leer()	readln()
Retornar	reddere	retornar	return

En la salida RISC-V, los decimales se representan con precisión simple de 32 bits, incluso cuando el tipo del lenguaje se llama double.

9. Ejemplos guiados

9.1 Probar polimorfismo con el ejemplo incluido

Abre testfile/main.pig. Sus importaciones cargan las clases Impresora, ImpresoraPDF, ImpresoraExcel e ImpresoraRed. Revisa los tipos explícitos de las variables y las llamadas al método imprimir.

Compila y ejecuta. Comprueba en consola que las llamadas utilizan las implementaciones de las clases concretas. Después consulta la tabla de tipos y las cuartetas para relacionar las clases con la traducción generada.

9.2 Probar la entrada de consola

Crea un archivo entrada.pig con:

```
VARIABILES>

esto numero : numerus 0;

MAIOR>

>> "Escribe un numero: ";

numero <<

>> "Valor recibido: " >> numero;

FINIS;
```

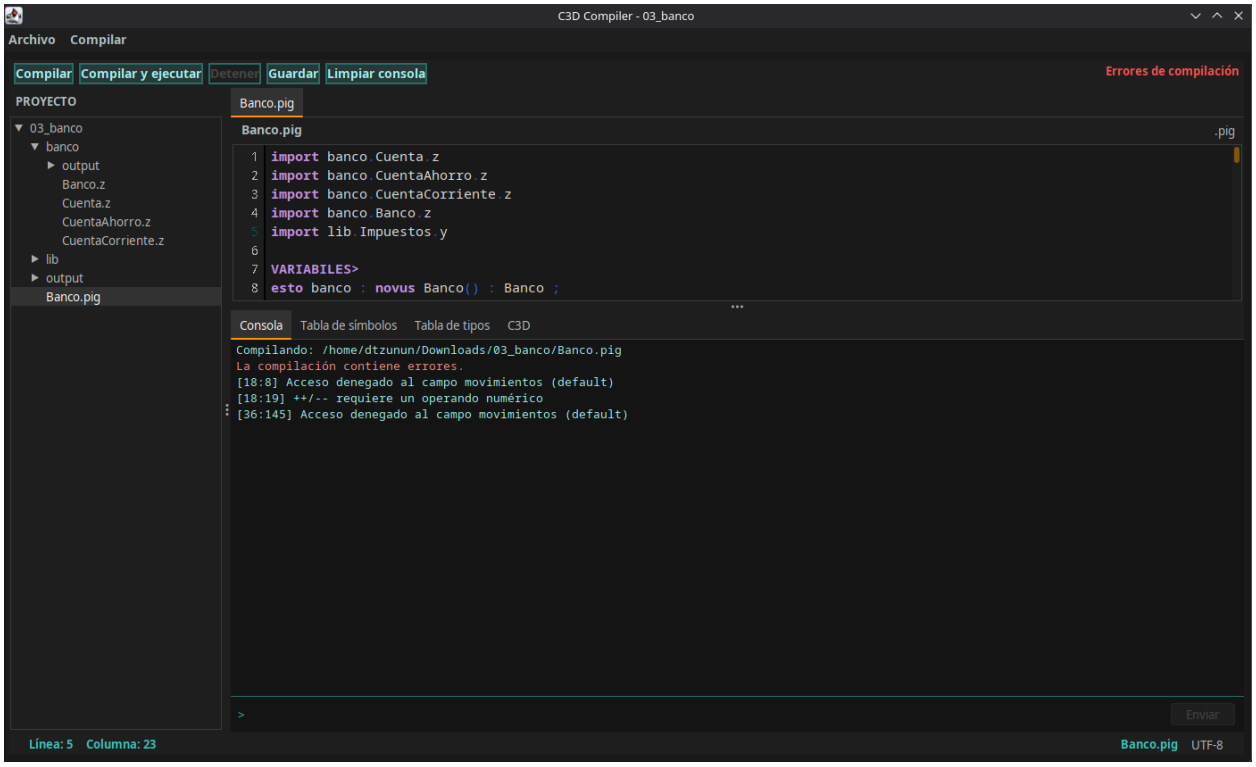
Pulsa **Compilar y ejecutar**, escribe 25 en el campo inferior y pulsa Enter. El programa debe mostrar el valor recibido y finalizar.

10. Errores frecuentes

Los mensajes concretos pueden incluir información adicional del contexto.

Mensaje o situación	**Causa probable**	**Solución**
Falta FINIS o aparece missing	Programa Lat incompleto	Revisa el cierre FINIS; y los cierres de bloques

Mensaje o situación	**Causa probable**	**Solución**
no viable alternative	Sintaxis incorrecta	Revisa la línea indicada y la indentación si es Y
Tipo no definido	Clase o estructura no cargada	Revisa las importaciones del .pig
Variable duplicada	Nombre repetido en el ámbito	Cambia el nombre o elimina la declaración repetida
No se puede asignar double a int	Conversión que pierde precisión	Usa un tipo decimal o modifica la expresión
La condición debe ser booleana	Se utilizó un número o cadena como condición	Escribe una comparación o una expresión booleana
No se pudo cargar el módulo	Ruta de importación incorrecta	Verifica la ruta relativa al .pig y la extensión
Acceso denegado	Miembro o constructor no accesible	Revisa su modificador y el lugar de la llamada
El método requiere @Override	Sobrescritura sin anotación	Añade @Override y conserva la firma
@Override no corresponde a un método heredado	No coincide con un método del padre	Revisa el nombre y los parámetros
El método sobrescrito reduce el acceso	La versión hija es menos visible	Mantén o amplía la visibilidad
La clase base no tiene constructor sin argumentos	Falta una llamada explícita al padre	Coloca super(argumentos); al inicio del constructor
super(...) debe ser la primera instrucción	La llamada aparece después de otra sentencia	Muévela al comienzo del constructor
No se encontró lib/rars1_6.jar	Falta el simulador	Recupera el JAR en la carpeta lib/ del proyecto
RARS informa un error	Problema al ensamblar, ejecutar o leer un dato	Revisa el mensaje de consola y los datos enviados
El programa parece detenido	Está esperando entrada o permanece en un ciclo	Envía el dato solicitado o pulsa Detener



11. Preguntas frecuentes

¿Qué archivo compilo si tengo varios?
El .pig principal, que debe importar los módulos Y y Z necesarios.

¿Puedo compilar un .y o .z solo?
Sí. El servicio admite ambas extensiones. Para ejecutar el comportamiento de objetos Z normalmente se utiliza un programa Lat que los construya y llame a sus métodos.

¿Dónde queda el ensamblador?
En output/, junto al archivo fuente compilado, con el mismo nombre base y extensión .s.

¿Puedo ver las cuartetas?
Sí, en la pestaña C3D después de una compilación correcta.

¿Cómo ejecuto el programa desde la aplicación?
Con Compilar y ejecutar o F5. La salida aparece en Consola.

¿Dónde escribo las respuestas que pide el programa?
En el campo inferior de Consola. Enter o Enviar remiten el dato al programa.

¿Cómo interrumpo una ejecución?
Con Detener. Limpiar consola únicamente borra el texto mostrado.

¿Z permite herencia y polimorfismo?
Sí. Admite herencia simple, sobrescritura obligatoriamente anotada y selección dinámica del método según la clase real del objeto.

¿Un miembro sin modificador es público?
No. En este proyecto se comporta como privado.

¿Necesito abrir RARS por separado?
La interfaz inicia el simulador automáticamente cuando utilizas Compilar y ejecutar. También puedes ejecutar el .s desde una terminal con el JAR incluido.